

Setting Up Prisma ORM

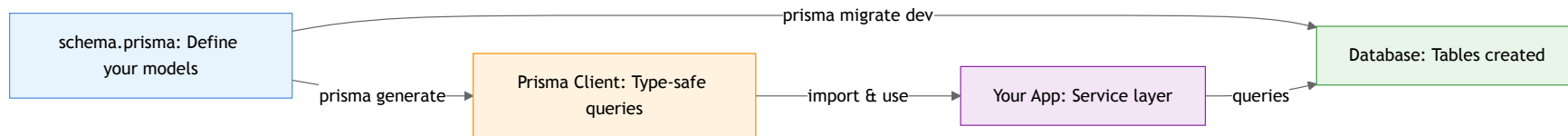
Adding a real database to your Node.js + Express + TypeScript API

What is Prisma?

- **Next-generation ORM** for Node.js and TypeScript
- **Type-safe** – your queries are validated at compile time
- Generates a **client** from your schema – no raw SQL needed
- Handles **migrations** – keeping your database and code in sync
- Includes **Prisma Studio** – a visual database browser

Prisma version 6.x · Imports from `@prisma/client`

How Prisma Works



One schema file drives everything – migrations, the generated client, and TypeScript types.

1 – Install Prisma

```
npm install prisma --save-dev
npm install @prisma/client
```

Package	Purpose
<code>prisma</code>	CLI – migrations, generate, studio (dev only)
<code>@prisma/client</code>	Runtime query engine used in your code

Why `--save-dev` **for** `prisma` ? The `prisma` package is only the CLI – you use it to write migrations and generate the client during development, but it's never imported by your running app. `@prisma/client` is what your app actually calls at runtime, so it's a regular dependency.

2 – Initialise Prisma

```
npx prisma init --datasource-provider postgresql
```

Creates two files:

```
prisma/  
  schema.prisma  ← your data model lives here  
  .env           ← DATABASE_URL environment variable
```

`.env` contents:

```
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/mydb"
```

Add `.env` to your `.gitignore` – never commit credentials

3 – Define Your Schema

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}  
  
model Author {  
  id      Int      @id @default(autoincrement())  
  name    String  
  email   String   @unique  
  posts   Post[]  
}
```

3 – Define Your Schema

```
model Post {  
  id      Int      @id @default(autoincrement())  
  title   String  
  content String?  
  published Boolean @default(false)  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
  author   Author   @relation(fields: [authorId], references: [id])  
  authorId Int  
}
```

Schema Concepts

Syntax	Meaning
<code>@id</code>	Primary key
<code>@default(autoincrement())</code>	Auto-incrementing integer
<code>@default(now())</code>	Set to current timestamp on insert
<code>@updatedAt</code>	Auto-updated on every save
<code>@unique</code>	Unique constraint on the column
<code>@relation(fields, references)</code>	Foreign key relationship
<code>String?</code>	Optional / nullable field

4 – Run Your First Migration

```
npx prisma migrate dev --name init
```

This does three things:

1. Creates `prisma/migrations/` with SQL migration files
2. Applies the migration to your PostgreSQL database
3. Automatically runs `prisma generate`

Every schema change → new migration

```
# Example: after adding a new field  
npx prisma migrate dev --name add-post-slug
```

Migration Workflow

Development

```
# Prototype quickly (no migration file)
npx prisma db push

# Create a real migration
npx prisma migrate dev --name <name>

# Reset everything
npx prisma migrate reset
```

When to use each

Command	Use When
db push	Prototyping, throwaway changes
migrate dev	Real features, team projects
migrate reset	Starting fresh locally

5 – Generate the Prisma Client

```
npx prisma generate
```

Generates a **type-safe client** into `node_modules/@prisma/client`

```
// Import in your code  
import { PrismaClient } from "@prisma/client";
```

`migrate dev` runs this automatically – only run it manually if you need to regenerate without a migration

6 – Prisma Client Singleton

Create `src/prismaClient.ts` :

```
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export default prisma;
```

Import and use it across your app:

```
import prisma from "../prismaClient.js";

const posts = await prisma.post.findMany();
```

One instance = one connection pool shared across the whole app

7 – Service Layer (Read)

Replace the in-memory array from Part 4:

```
import prisma from "../prismaClient.js";
import type { Post } from "@prisma/client";

export class PostService {
  async findAll(): Promise<Post[]> {
    return prisma.post.findMany({
      include: { author: true },
    });
  }

  async findById(id: number): Promise<Post | null> {
    return prisma.post.findUnique({
      where: { id },
      include: { author: true },
    });
  }
}
```

7 – Service Layer (Write)

```
async create(data: {
  title: string;
  content?: string;
  authorId: number;
}): Promise<Post> {
  return prisma.post.create({ data, include: { author: true } });
}

async update(
  id: number,
  data: { title?: string; content?: string; published?: boolean },
): Promise<Post | null> {
  const existing = await prisma.post.findUnique({ where: { id } });
  if (!existing) return null;
  return prisma.post.update({ where: { id }, data, include: { author: true } });
}

async delete(id: number): Promise<boolean> {
  const existing = await prisma.post.findUnique({ where: { id } });
  if (!existing) return false;
  await prisma.post.delete({ where: { id } });
  return true;
}
```

What Changed from Part 4?

Part 4 – In-Memory

Prisma

```
this.posts.find(p => p.id === id)
```

```
prisma.post.findUnique({ where: { id } })
```

```
this.posts.push(newPost)
```

```
prisma.post.create({ data })
```

```
Object.assign(post, data)
```

```
prisma.post.update({ where, data })
```

```
this.posts.splice(index, 1)
```

```
prisma.post.delete({ where: { id } })
```

Manual `id` increment

```
@default(autoincrement())
```

Data lost on restart

Data persisted in PostgreSQL

The controller and routes **don't change** – the service is the only layer that knows about the database.

Now it's your turn

Exercise 7 – Add Prisma to your expenses API: define an a schema, run a migration, create the Prisma singleton, and create a service to get data via Prisma.